

作業ログ

報告者：山口汐里

報告日：2026 年 5 月 26 日

プロジェクト名：受動的環境介入によるメンタルヘルス支援：大学空間における自動演奏装置の提案

1. 進捗サマリー

- 全体進捗率：65%（音声合成フェーズ）
 - マイルストーン達成状況：進行中
 - 主要成果：
 - Audacity を用いたサンプル音声（C4）の FFT 解析完了，倍音比率データを取得
 - Processing + Minim ライブラリでフルート音声合成器（flute2.pde / flute3.pde）を実装
 - 試作音声とサンプル音声のスペクトル比較により，倍音比率・LPF カットオフを修正
 - 重大課題（影響度：高）：
 - なし（高次倍音の精度に軽微な改善余地あり）
-

2. プロジェクト計画（ガントチャート）

図 1 にプロジェクト全体のガントチャートを示す．今週（5/20～5/26）は WBS 2.1.2「フルート音色合成（倍音 +ADSR）」に相当し，第 6～8 週の前半にあたる．本週の作業はスケジュール通りに完了した．



図1 Arduino オーケストラ プロジェクトガントチャート

3. 作業ログ

作業項目	開始日	予定完了	状態	進捗率	コメント
フルート倍音理論調査	5/20	5/20	完了	100%	フーリエ変換・倍音構造を参考文献で確認 [1, 2, 3, 4]
サンプル音声 FFT 解析	5/20	5/21	完了	100%	Audacity でサンプル音声 (C4) を解析, 各倍音の相対振幅を記録
設計方針の確定	5/21	5/21	完了	100%	設計ワークシートに調査結果を記入, ADSR・LPF・ノイズ成分の方針を確定 [1, 3, 4]
flute2.pde 実装	5/22	5/22	完了	100%	FluteInstrument クラス・8倍音ウェーブテーブル生成, AI 支援を併用
試作音声生成・スペクトル比較	5/23	5/23	完了	100%	Audacity / WavePad でスペクトル比較, 第2倍音の過剰を確認・スクショ保存
flute3.pde 改良実装	5/24	5/25	完了	100%	倍音比率全体を下方修正, ビブラート追加 (4.3 Hz・7.4 セント), LPF を基音×6 に変更
聴感テスト・最終確認	5/25	5/25	完了	100%	試作音声を耳で確認, WavePad でスペクトルを再比較・スクショ保存
作業ログ作成	5/26	5/26	完了	100%	本ドキュメントの作成

4. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
高次倍音の精度不足	実測 FFT 値とウェーブテーブル係数の微少な差異	低	低	次フェーズで係数をさらに調整	未定	対応中

5. AI 利用状況と検証

5.1 利用内容

- 利用目的：コード設計・実装支援
- 入力（プロンプト要旨）：
 - Processing + Minim ライブラリを用いたフルート音声合成のクラス設計方針を相談
 - `FluteInstrument` クラスの構造（二重 ADSR 方式の是非）を確認
 - ADSR パラメータの初期値・`WavetableGenerator.gen10` の係数設定を質問
- 出力概要：
 - `FluteInstrument` の基本構造（`toneEnv` / `breathEnv` の独立した ADSR 管理）を得た
 - ウェーブテーブル係数を FFT 測定値に近づける手順を確認した

5.2 検証方法

- 検証手法：
 - 生成音声とサンプル音声の FFT スペクトルを目視で比較
- 参照資料：
 - フルートの音作成の確認 [1]
 - ADSR のバランスについての確認 [3] [4]
- 検証手順：
 1. サンプル音声（C4）を Audacity で開き FFT 表示
 2. 試作音声を WavePad でスペクトル表示し、同一スケールに設定
 3. 両者を並べてスクリーンショット保存
 4. 各倍音ピークの相対強度を目視で比較し、ウェーブテーブル係数を修正

5.3 ファクトチェック結果

- 一致点：

- 二重 ADSR 構成（本体音・呼気ノイズ別管理）は参考文献 [1] の設計方針と整合
- ADSR のアタック時間帯にノイズを強調する手法も文献と一致
- 不一致・疑義：
 - AI 提案の LPF カットオフ（基音×9）はやや高く，金属的な音質が生じた
- 最終判断：
 - 部分採用（クラス構造・ADSR 設計は採用，LPF カットオフは実測に基づき基音×6 に独自修正）
- 根拠：
 - WavePad スペクトル比較の目視結果，および聴感テストで金属感の低減を確認

6. 次回計画

- 目標：
 - さらなる音質改善（高次倍音の精度向上・ノイズ成分の自然化）
- 達成基準：
 - サンプル音声の FFT スペクトルと試作音声のスペクトルが目視で概ね一致すること
 - 聴感テストで「フルートらしさ」に問題がないと判断できること
- 期限：
 - 未定

7. 評価基準

今回の評価指標を以下に示す。現時点では音声合成フェーズ（2 系統）が進行中であり，評価テストは第 8～10 週に実施予定。

WBS	評価項目	担当	実施時期	達成基準
3.1	単体・周期精度テスト（V1 音程 / V2 FFT 比較）	全員	第 8～9 週	スペクトル目視一致・音程誤差許容範囲内
3.2	全体合奏演奏テスト	全員	第 9～10 週	合奏時間 120 秒以上の連続動作
3.3	可変テンポ動作検証	全員	第 10 週	テンポ誤差 ± 50 ms 以内

本週（WBS 2.1.2）に対する自己評価：

- FFT 実測値と合成スペクトルの目視比較で概ね一致を確認した。
- LPF カットオフを AI 提案値（基音×9）から実測ベース（基音×6）に独自修正し，金属感を低減
- 高次倍音の微小誤差は残存しており，3.1 の正式評価前に追加調整が必要

8. その他

- 使用ソフトウェア：Audacity（FFT 解析・スペクトル比較），WavePad（スペクトル表示・スクリーンショット）
 - スクリーンショットは各試作ごとにプロジェクトルートへ保存済み
-

9. 付録

- 添付範囲：
 - 差分（flute3.pde は flute2.pde からの変更点が主体）

Listing 1 flute2.pde

```
1 import ddf.minim.*;
2 import ddf.minim.ugens.*;
3
4 Minim minim;
5 AudioOutput out;
6 Waveform currentWaveform;
7
8 String[] melody = {
9     "C4", "C4", "G4", "G4", "A4", "A4", "G4"
10 };
11
12 float[] duration = {
13     1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 1.0f
14 };
15
16 float[] startTime = {
17     0.0f, 1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f
18 };
19
20 class FluteInstrument implements Instrument
21 {
22     ADSR toneEnv;
23     ADSR breathEnv;
24
25     FluteInstrument(float frequency, float maxAmp, Waveform wf)
26     {
27         float cutoff = (float)Math.min(18000.0, frequency * 9.0);
28
29         Oscil tone = new Oscil(frequency, 1.0f, wf);
30         MoogFilter toneLPF = new MoogFilter(cutoff, 0.0f, MoogFilter.Type.LP);
31
32         Noise breath = new Noise(1.0f, Noise.Tint.WHITE);
33         MoogFilter breathBand = new MoogFilter(2600.0f, 0.25f, MoogFilter.Type.BP);
34
35         toneEnv = new ADSR(maxAmp, 0.06f, 0.14f, 0.82f, 0.16f);
36         breathEnv = new ADSR(maxAmp * 0.025f, 0.02f, 0.12f, 0.18f, 0.10f);
```

```

37
38     tone.patch(toneLPF);
39     toneLPF.patch(toneEnv);
40
41     breath.patch(breathBand);
42     breathBand.patch(breathEnv);
43 }
44
45 void noteOn(float duration)
46 {
47     toneEnv.patch(out);
48     breathEnv.patch(out);
49
50     toneEnv.unpatchAfterRelease(out);
51     breathEnv.unpatchAfterRelease(out);
52
53     toneEnv.noteOn();
54     breathEnv.noteOn();
55 }
56
57 void noteOff()
58 {
59     toneEnv.noteOff();
60     breathEnv.noteOff();
61 }
62 }
63
64 void setup()
65 {
66     size(512, 220);
67     minim = new Minim(this);
68     out = minim.getLineOut();
69     out.setTempo(120);
70
71     currentWaveform = makeFluteWaveform();
72 }
73
74 Wavetable makeFluteWaveform()
75 {
76     Wavetable wf = WavetableGenerator.gen10(
77         8192,
78         new float[] {
79             1.00f, 0.68f, 0.21f, 0.28f,
80             0.20f, 0.08f, 0.07f, 0.07f
81         }
82     );
83
84     wf.normalize();
85     wf.scale(0.9f);
86     return wf;
87 }
88
89 void playSong()
90 {

```

```

91     out.pauseNotes();
92
93     for (int i = 0; i < melody.length; i++)
94     {
95         out.playNote(
96             startTime[i],
97             duration[i],
98             new FluteInstrument(
99                 Frequency.ofPitch(melody[i]).asHz(),
100                 0.45f,
101                 currentWaveform
102             )
103         );
104     }
105
106     out.resumeNotes();
107 }
108
109 void draw()
110 {
111     background(0);
112     stroke(255);
113
114     for (int i = 0; i < out.bufferSize() - 1; i++)
115     {
116         line(i, 70 - out.left.get(i) * 60,
117             i + 1, 70 - out.left.get(i + 1) * 60);
118
119         line(i, 160 - out.right.get(i) * 60,
120             i + 1, 160 - out.right.get(i + 1) * 60);
121     }
122
123     fill(255);
124
125 }
126
127 void keyPressed()
128 {
129     switch(key)
130     {
131         case 'p':
132             playSong();
133             break;
134
135         case '1':
136             currentWaveform = makeFluteWaveform();
137             break;
138
139     }
140 }
141
142 void stop()
143 {
144     minim.stop();

```



```

145     super.stop();
146 }

```

Listing 2 flute3.pde

```

1  import ddf.minim.*;
2  import ddf.minim.ugens.*;
3
4  Minim minim;
5  AudioOutput out;
6  Waveform currentWaveform;
7
8  String[] melody = {
9      "C4", "C4", "G4", "G4", "A4", "A4", "G4"
10 };
11
12 float[] duration = {
13     2.0f, 2.0f, 2.0f, 2.0f, 2.0f, 2.0f, 2.0f
14 };
15
16 float[] startTime = {
17     0.0f, 2.0f, 4.0f, 6.0f, 8.0f, 10.0f, 12.0f
18 };
19
20 class FluteInstrument implements Instrument
21 {
22     ADSR toneEnv;
23     ADSR breathEnv;
24
25     FluteInstrument(float frequency, float maxAmp, Waveform wf)
26     {
27         float cutoff = (float)Math.min(9000.0, frequency * 6.0);
28
29         Oscil tone = new Oscil(frequency, 1.0f, wf);
30
31         float vibratoRate = 4.3f;
32         float vibratoCent = 7.4f;
33         float vibratoDepthHz = frequency * (pow(2, vibratoCent / 1200.0f) - 1.0f);
34
35         Oscil vibrato = new Oscil(vibratoRate, vibratoDepthHz, Waves.SINE);
36         vibrato.offset.setLastValue(frequency);
37         vibrato.patch(tone.frequency);
38
39         MoogFilter toneLPF = new MoogFilter(cutoff, 0.0f, MoogFilter.Type.LP);
40
41         Noise breath = new Noise(1.0f, Noise.Tint.WHITE);
42         MoogFilter breathBand = new MoogFilter(2600.0f, 0.25f, MoogFilter.Type.BP);
43
44         toneEnv = new ADSR(maxAmp, 0.06f, 0.14f, 0.82f, 0.16f);
45         breathEnv = new ADSR(maxAmp * 0.05f, 0.015f, 0.12f, 0.12f, 0.10f);
46
47         tone.patch(toneLPF);
48         toneLPF.patch(toneEnv);

```

```

49
50     breath.patch(breathBand);
51     breathBand.patch(breathEnv);
52 }
53
54 void noteOn(float duration)
55 {
56     toneEnv.patch(out);
57     breathEnv.patch(out);
58
59     toneEnv.unpatchAfterRelease(out);
60     breathEnv.unpatchAfterRelease(out);
61
62     toneEnv.noteOn();
63     breathEnv.noteOn();
64 }
65
66 void noteOff()
67 {
68     toneEnv.noteOff();
69     breathEnv.noteOff();
70 }
71 }
72
73
74 void setup()
75 {
76     size(512, 220);
77     minim = new Minim(this);
78     out = minim.getLineOut();
79     out.setTempo(120);
80
81     currentWaveform = makeFluteWaveform();
82 }
83
84 Wavetable makeFluteWaveform()
85 {
86     Wavetable wf = WavetableGenerator.gen10(
87         8192,
88         new float[] {
89             1.00f, 0.35f, 0.14f, 0.12f,
90             0.08f, 0.04f, 0.025f, 0.02f
91         }
92     );
93
94     wf.normalize();
95     wf.scale(0.75f);
96     return wf;
97 }
98
99 void playSong()
100 {
101     out.pauseNotes();
102

```

```

103   for (int i = 0; i < melody.length; i++)
104   {
105       out.playNote(
106           startTime[i],
107           duration[i],
108           new FluteInstrument(
109               Frequency.ofPitch(melody[i]).asHz(),
110               0.45f,
111               currentWaveform
112           )
113       );
114   }
115
116   out.resumeNotes();
117 }
118
119 void draw()
120 {
121     background(0);
122     stroke(255);
123
124     for (int i = 0; i < out.bufferSize() - 1; i++)
125     {
126         line(i, 70 - out.left.get(i) * 60,
127             i + 1, 70 - out.left.get(i + 1) * 60);
128
129         line(i, 160 - out.right.get(i) * 60,
130             i + 1, 160 - out.right.get(i + 1) * 60);
131     }
132
133     fill(255);
134
135 }
136
137 void keyPressed()
138 {
139     switch(key)
140     {
141         case 'p':
142             playSong();
143             break;
144
145         case '1':
146             currentWaveform = makeFluteWaveform();
147             break;
148
149     }
150 }
151
152 void stop()
153 {
154     minim.stop();
155     super.stop();
156 }

```

参考文献

- [1] Dirigent, 「Rob Papen PREDATOR でフルートサウンドを作ってみた」, Dirigent Blog, <https://dirigent.jp/blog/predator-flute>, 2026 年 5 月 26 日閲覧.
- [2] 川崎 優生奈, 田村 仁, 「拡散モデルと倍音の特徴を用いた音源分離手法の検討」, 日本工業大学機械システム工学専攻, FIT2024 (第 23 回情報科学技術フォーラム), CE-007.
- [3] SONICWARE, 「シンセサイザーで「音作り」をするための基礎知識」, SONICWARE Blog, <https://sonicwire.com/news/blog/2024/04/synthesizer>, 2026 年 5 月 26 日閲覧.
- [4] EFFECTOR'S LAB, 「コンプレッサー入口編③ アタック完全ガイド」, EFFECTOR'S LAB, <https://effecters-lab.com/2025/08/17/comp-3/>, 2026 年 5 月 26 日閲覧.